

从命令行到浏览器：XATS2JS 在线编译环境的设计与实现

摘要

ATS 是一种强调类型安全与程序验证的函数式编程语言，其依赖类型、线性类型等机制能够在编译阶段描述并检查较为复杂的程序性质。然而，ATS 工具链通常依赖本地命令行环境，对初次接触该语言的用户存在一定使用门槛。基于 ATS 语言社区在 GitHub 上公开的 XATS2JS 项目，我实现了一个运行于浏览器中的 ATS/Xanadu 到 JavaScript 编译环境，并为其开发了可视化网站。用户可以直接在网页中输入 ATS 代码，查看编译生成的 JavaScript、原始输出与错误诊断，也可以浏览编译器依赖的 Prelude 文件。本文主要记录该项目的设计思路、核心实现、开发过程中遇到的问题，以及当前系统的局限。

关键词：ATS；Xanadu；XATS2JS；编译器；JavaScript；浏览器

一、项目背景与目标

ATS 在类型系统和程序验证方面具有较强的表达能力，但其安装、环境变量配置以及命令行编译流程可能阻碍初学者进行快速实验。传统在线编译器通常通过服务器接收代码，再调用后端编译工具；这种方式需要维护服务器环境，也会产生额外的部署和通信成本。因此，本项目尝试将 XATS2JS 编译过程直接迁移到浏览器，使用户无需安装 ATS 工具链即可完成简单代码的编译。

项目的主要目标包括：在浏览器中运行原本依赖 Node.js 环境的 XATS2JS 编译器；建立可供编译器读取的虚拟文件系统；加载 ATS Prelude 标准库文件；捕获编译器的标准输出与错误信息；最后通过 React 网站提供代码输入、结果展示和 Prelude 浏览功能。

二、系统结构与编译流程

整个系统可以分为编译器、浏览器兼容层、文件资源层和用户界面四个部分。编译器部分使用 ATS 社区项目生成的 JavaScript 编译器文件；兼容层负责模拟其需要的 Node.js 全局对象；文件资源层保存用户代码和 Prelude；用户界面则负责组织编译操作及结果展示。

用户点击“Compile”按钮后，网页首先把输入内容以`input.dats`为文件名注册到内存文件表中。随后，程序构造类似命令行环境的`process.argv`，将该文件作为编译器输入，并调用 XATS2JS 生成代码中的编译入口。编译期间产生的内容分别写入模拟的`stdout`和`stderr`对象。编译结束后，网页提取生成的 JavaScript，同时显示错误和诊断信息。

其基本流程可以概括为：

输入 ATS 代码

- 注册到虚拟文件系统
- 构造编译参数
- 调用 XATS2JS
- 捕获输出和异常
- 整理并展示 JavaScript

为了提高输出的可读性，我还增加了简单的后处理步骤，包括过滤部分注释、合并多余空行，以及简化某些临时变量后立即返回的代码结构。网页同时保留原始标准输出，便于调试并对比处理前后的结果。

三、浏览器运行环境的适配

该项目最主要的困难不是页面设计，而是编译器运行环境的迁移。XATS2JS 虽然已经被编译为 JavaScript，但生成的程序仍然假设自己运行在 Node.js 中，会访问 ``process``、``require``、文件系统以及标准输出。浏览器并不原生提供这些接口，因此不能直接加载并运行。

为解决这一问题，我实现了一层浏览器兼容代码。该层在 ``globalThis`` 上建立运行时对象，内部包含文件映射表、标准输出、标准错误以及文件注册和读取方法。同时，我为 ``require("fs")`` 提供了受限模拟，使编译器能够通过 ``readFileSync`` 和 ``accessSync`` 访问内存中的文件。对于 XATS 使用的路径，还需要处理 ``$(XATSHOME)`` 和 ``prelude/`` 等前缀，使同一文件能够以不同形式被查找到。

Prelude 文件数量较多，不适合逐个手工引入。因此，我编写了文件映射生成工具，遍历 Prelude 目录，将文件路径和文本内容转换成可由前端导入的 JavaScript 数据。网页初始化时会把这些文件统一注册到虚拟文件系统，从而为编译器提供相对完整的基础环境。

四、网站功能设计

网站使用 React 与 Vite 构建，并通过路由划分为编译器和 Prelude Browser 两个主要页面。编译器页面采用输入区与输出区并列的形式：左侧输入 ATS/Xanadu 源代码，右侧显示 JavaScript 结果、原始输出和诊断信息。这种布局便于用户观察源程序与编译结果之间的对应关系。

Prelude Browser 会根据文件路径动态构造目录树。用户可以展开目录、选择文件，并在页面中阅读其内容。该功能不仅用于展示资源，也有助于学习 ATS 基础定义以及排查编译过程中缺失文件的问题。

由于编译在用户浏览器中完成，系统不需要将源代码上传到服务器。这既简化了部署结构，也使项目能够作为一个相对独立的静态网站运行。

五、开发体会与项目局限

这次开发使我认识到，将编译器“编译为 JavaScript”并不等于它能够自然地运行在浏览器中。真正的迁移工作主要发生在运行时边界：文件从哪里读取、命令行参数如何传递、输出写向何处，以及原有程序依赖的宿主接口如何替代。相比重新实现编译逻辑，为既有编译器建立一个足够小而明确的兼容层，是本项目较为关键的设计选择。

目前系统仍存在一些局限。例如，虚拟文件系统主要面向内置 Prelude 和单个输入文件；输出整理依赖文本规则，不能替代正式的代码优化；编辑器也暂未提供语法高亮、自动补全和源码级错误定位。未来可以进一步支持多文件工程、用户文件导入、代码在线执行以及更加结构化的编译诊断。

六、结语

本项目在 ATS 社区已有工作的基础上，将 XATS2JS 从命令行运行方式迁移到了浏览器，并完成了一个集代码输入、编译结果展示、错误诊断和 Prelude 浏览于一体的 Web 工具。它不仅是一次前端网站开发，也是一项关于编译器运行环境迁移、接口兼容与资源组织的工程实践。通过这一项目，我对 ATS 工具链的结构以及浏览器与 Node.js 运行环境之间的差异有了更加具体的理解。